CANVAS SCREEN PLUGIN | 480×272 CAPACITIVE TOUCH

DeckX Canvas Screen Plugin Developer Guide

Full-Screen Interactive SkiaSharp Dashboards & Touch Controls Specification

Interface: `IDeckXCanvasPlugin`

Target Hardware: DeckX Control Pad ESP32 (480×272 Touch Display)

NuGet Package: `DeckX.Core` v1.0.0

1. Overview

DeckX Canvas Plugins allow developers to transform the DeckX Control Pad's 480×272 screen into a **dynamic interactive dashboard**. Unlike traditional button plugins that render inside individual keypad slots, a Canvas Plugin has **complete ownership** over the full 480×272 SkiaSharp drawing surface and receives **capacitive touch coordinates** directly from the hardware screen.

Core Capabilities:

- **Full-Screen 2D Graphics:** Render gauges, charts, audio visualizers, Spotify album covers, and custom UI components using SkiaSharp.
- **Capacitive Multi-Zone Hit-Testing:** Detect taps, button toggles, slider drags, and +/- increment controls with `SKRect.Contains(x, y)`.
- **Event-Driven Immediate Redraws:** Fire `RequestRedraw?.Invoke()` to update the hardware display immediately upon user interaction with near-zero latency.
- **Rotary Encoder Integration:** Receive hardware knob scroll deltas via `OnEncoderRotate(index, delta)`.

> ⚡ **1-Click Auto-Packer:**
> Building your project with `dotnet build -c Release` compiles your code, zips all assets into a `.deckxplugin` bundle, and automatically deploys it into `%APPDATA%\DeckX\Plugins\<id>\`.

2. NuGet Installation & `.csproj` Configuration

Create a .NET 8.0 Class Library project and install the official NuGet packages:

```
dotnet new classlib -n MyCanvasPlugin -f net8.0
dotnet add package DeckX.Core --version 1.0.0
dotnet add package SkiaSharp --version 2.88.9
```

Project File (`.csproj`) Setup:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <CopyLocalLockFileAssemblies>true</CopyLocalLockFileAssemblies>
  </PropertyGroup>

  <!-- Reference DeckX.Core from NuGet for standalone developer builds -->
  <ItemGroup Condition="!Exists('..\..\DeckX.Core\DeckX.Core.csproj')">
    <PackageReference Include="DeckX.Core" Version="1.0.0">
      <PrivateAssets>all</PrivateAssets>
      <ExcludeAssets>runtime</ExcludeAssets>
    </PackageReference>
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="SkiaSharp" Version="2.88.9" />
  </ItemGroup>

</Project>
```

3. The `IDeckXCanvasPlugin` Interface

Defined in `DeckX.Core.Plugins`, implement this interface in your main dashboard class:

```
using System;
using SkiaSharp;

namespace DeckX.Core.Plugins;

public interface IDeckXCanvasPlugin
{
    string Id { get; }            // e.g. "com.deckx.canvas.sampletemplate"
    string Name { get; }          // e.g. "DeckX Canvas Plugin Template"
    string Author { get; }        // Developer or Studio name
    string Version { get; }       // e.g. "1.0.0"
    string Description { get; }   // Summary of dashboard functionality
    string Category { get; }      // e.g. "Custom Dashboards"
    string? CategoryIconGeometry { get; } // SVG path string

    // Target FPS: 0-1 for event-driven dashboards, or 10-20 FPS for live meters
    int TargetFps => 1;

    // When true, DeckX continuously redraws canvas at TargetFps
    // When false, DeckX redraws on-demand when RequestRedraw is triggered
    bool NeedsContinuousStream => false;

    // Trigger this event to immediately redraw on PC and ESP32 hardware
    event Action? RequestRedraw;

    // Render the 480x272 SkiaSharp scene
    void RenderCanvas(SKCanvas canvas, int width, int height);

    // Touch input handlers from physical capacitive screen
    void OnTouchDown(int x, int y) { }
    void OnTouchMove(int x, int y) { }
    void OnTouchUp(int x, int y) { }

    // Optional rotary encoder knob rotation events
    void OnEncoderRotate(int encoderIndex, int delta) { }

    // Lifecycle
    void OnInitialize(string packageDirectory) { }
    void OnShutdown() { }
}
```

4. Capacitive Touch Hit-Testing Architecture

To make interactive buttons and sliders on the 480×272 screen, define pre-calculated `SKRect` bounding boxes for each interactive widget and test touches in `OnTouchDown`:

```
User taps physical screen (e.g. x: 50, y: 100) │ ▼ DeckX Firmware sends touch packet via USB Serial │ ▼ DeckX Studio invokes: plugin.OnTouchDown(50, 100) │ ├── HitRectToggle.Contains(50, 100) ➔ Toggle State (_isToggleOn = !_isToggleOn) ├── HitRectPlus.Contains(50, 100) ➔ Increment Counter (_counterValue++) └── HitRectTheme.Contains(50, 100) ➔ Cycle Theme Palette │ ▼ Plugin calls: RequestRedraw?.Invoke() │ ▼ DeckX re-renders canvas immediately and sends compressed RGB565 frame to ESP32!
```

Example Hit-Test Implementation:

```
private static readonly SKRect HitRectToggle = new(20, 56, 150, 150);
private static readonly SKRect HitRectMinus  = new(20, 164, 76, 224);
private static readonly SKRect HitRectPlus   = new(94, 164, 150, 224);

public void OnTouchDown(int x, int y)
{
    var pt = new SKPoint(x, y);

    if (HitRectToggle.Contains(pt))
    {
        _isToggleOn = !_isToggleOn;
        RequestRedraw?.Invoke();
    }
    else if (HitRectMinus.Contains(pt))
    {
        _counterValue = Math.Max(0, _counterValue - 1);
        RequestRedraw?.Invoke();
    }
    else if (HitRectPlus.Contains(pt))
    {
        _counterValue++;
        RequestRedraw?.Invoke();
    }
}
```

5. Full-Screen SkiaSharp 2D Rendering Walkthrough

In `RenderCanvas`, you are provided an `SKCanvas` surface measuring **480 pixels wide by 272 pixels high**.

```
public void RenderCanvas(SKCanvas canvas, int width, int height)
{
    // 1. Clear background
    canvas.Clear(new SKColor(15, 17, 23));

    // 2. Draw Top Navigation / Header Bar
    using (var headerPaint = new SKPaint { Color = SKColors.White, TextSize = 14, IsAntialias = true,
    {
        canvas.DrawText("SYSTEM MONITOR DASHBOARD", 20, 32, headerPaint);
    }

    // 3. Draw Toggle Card (Left Widget)
    DrawInteractiveCard(canvas, 20, 56, 130, 94, "MAIN POWER", _isToggleOn ? "ONLINE" : "STANDBY", _is

    // 4. Draw Counter (+/- Controls)
    DrawCounterWidget(canvas, 20, 164, 130, 60, _counterValue);

    // 5. Draw Radial Arc Gauge (Center Widget)
    DrawRadialArcGauge(canvas, 170, 56, 140, 168, "CPU LOAD", 72, SKColors.DeepSkyBlue);

    // 6. Draw Real-Time Waveform / Audio Graph (Right Widget)
    DrawWaveformGraph(canvas, 330, 56, 130, 168, _waveformData, SKColors.Cyan);
}

private void DrawInteractiveCard(SKCanvas canvas, float x, float y, float w, float h, string title, st
{
    using var bg = new SKPaint { Color = new SKColor(24, 27, 38), Style = SKPaintStyle.Fill, IsAntiali
    using var border = new SKPaint { Color = new SKColor(46, 52, 73), Style = SKPaintStyle.Stroke, Str
    canvas.DrawRoundRect(new SKRoundRect(new SKRect(x, y, x + w, y + h), 8, 8), bg);
    canvas.DrawRoundRect(new SKRoundRect(new SKRect(x, y, x + w, y + h), 8, 8), border);

    using var titlePaint = new SKPaint { Color = new SKColor(156, 163, 175), TextSize = 10, IsAntialia
    canvas.DrawText(title, x + 12, y + 22, titlePaint);

    using var statusPaint = new SKPaint { Color = statusColor, TextSize = 16, IsAntialias = true, Fake
    canvas.DrawText(status, x + 12, y + 54, statusPaint);
}
```

6. Manifest Configuration (`plugin.json`)

The `plugin.json` file tells DeckX that this assembly is a Canvas-mode dashboard:

```
{
  "id": "com.deckx.canvas.sampletemplate",
  "name": "DeckX Canvas Plugin Template",
  "version": "1.0.0",
  "author": "DeckX Developer",
  "description": "Reference developer template for creating full-screen 480x272 interactive canvas das
  "category": "Custom Dashboards",
  "assembly": "DeckX.Plugin.CanvasSampleTemplate.dll",
  "type": "canvas",
  "resolution": {
    "width": 480,
    "height": 272
  }
}
```

7. Performance Guidelines for 480×272 Displays

- **Use Event-Driven Rendering:** Set `NeedsContinuousStream = false` and `TargetFps = 1` unless rendering high-frequency waveforms. Only trigger redraws via `RequestRedraw` on touch or timer ticks.
- **Hit Area Padding:** Capacitive touch targets should be at least 44×44 pixels to ensure comfortable and accurate finger taps on hardware.
- **Dispose Resources:** If allocating bitmap textures or custom typefaces, ensure they are disposed or reused across render frames.